Unit 14 - Event Driven Programming
Unit 15 - Object Oriented Programming
Unit 22 - Developing Computer Games
Assignment 3 - Design and Implement an Application

Introduction

Scenario / Vocational Context - *As a member of your organisations' software development team you are required to design and develop a computer game in .NET C#. You have decided not to be too ambitious, and create a relatively simple game, but one that involves the use of both Event-Driven and Object Oriented techniques.*

You are to design, and develop, an Event Driven, Object Oriented computer game. This game is to be produced using MS .NET C#, and created using the Visual Studio IDE.

Your game should have a clear purpose, a way to score points, a way to win/lose, clear and detailed on-screen instructions for the user.

Your documentation should be comprehensive and include excellent in-code comments throughout.

Task 1 - Design *(Unit 14 P3; Unit 22 P3)*

Task description - *Please provide a set of design documentation as follows:*

- *Write a paragraph describing what your game will do, why the user will enjoy playing it, and what the purpose of the game is*
- *Provide an explanation of what inputs (eg mouse clicks, key-clicks) your program will handle*
- *Provide a storyboard showing the main processes/actions of the game*
- *Provide annotated screenshots showing the main graphical components in the game*
- *Describe one major event handler. Also provide a flowchart of this event handler.*
- *Describe how you will use a List, or similar collection, of objects.*

Write a paragraph describing what your game will do, why the user will enjoy playing it, and what the purpose of the game is

The video game that will be created will essentially be a strategic fighting game themed around historical weaponry. I believe that this may be enjoyable for the user due to the strategic thinking involved in which weapon to choose the strategic thinking required to win against the opponent. The intended purpose of the game is to select a weapon and win against the opponent, which will be an AI. The AI the player is playing against will have the same health and damage output as the player. Meaning that, depending on the use of the combination, it will either increase or decrease the chances of winning against the enemy or getting killed.

Provide an explanation of what inputs (eg mouse clicks, key-clicks) your program will handle

The featured inputs for the game will consist of almost exclusively mouse clicks, in order to interact with the different abilities and choices of the game. The reason for this is due to the game exclusively relying on the use off buttons to carry out all the functions and methods declared within the source code.

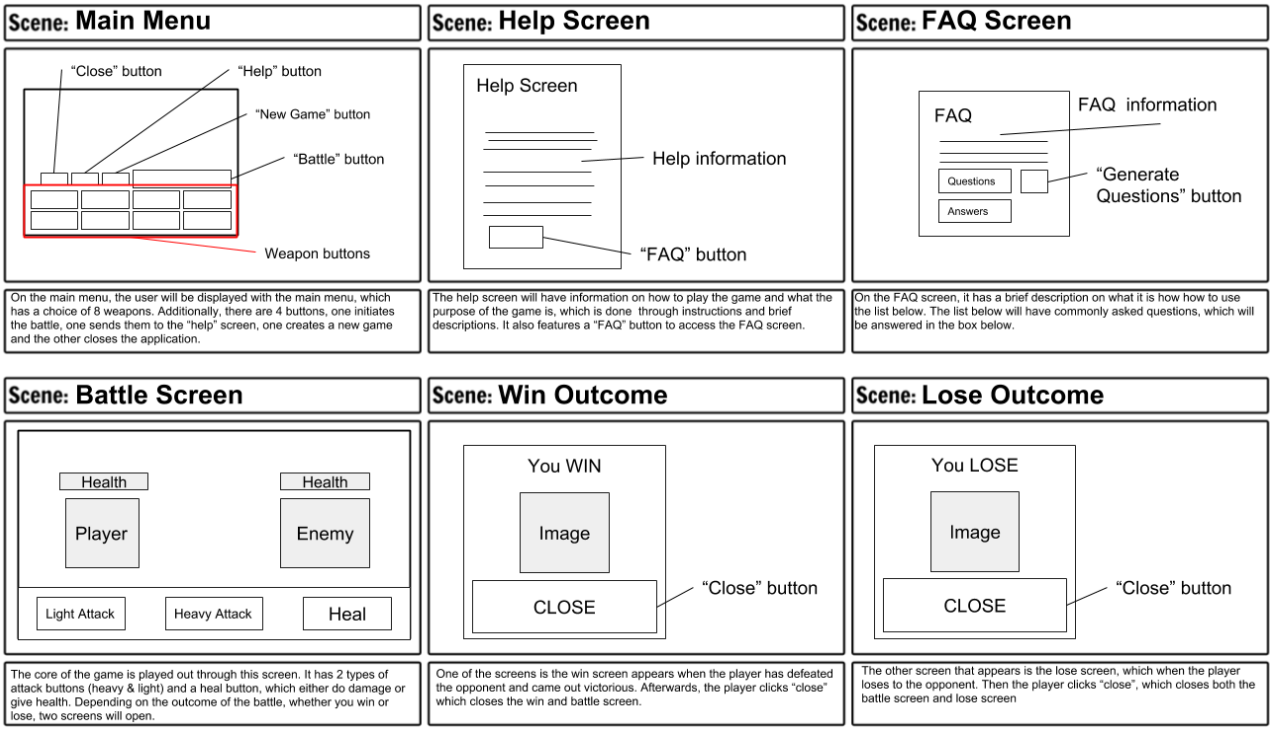
For instance, the mouse clicks will be used for vital event handlers. Such as buttons responsible for launching the different windows forms, and even the different types of attacks within the game. Essentially, mouse clicks is what is going to be the most used user input within the application, meaning that it is entirely reliable on the said input method.

For example, to access the battle form, the user will need to have the main menu open and, using the mouse and mouse button, clicks on the “Battle” button (input). The application will then use this registered input to reference and open the form linked to the button being clicked, which is the programme’s output from the said input.

Additionally, there are small input methods in which the keyboard can be used within the application, in the form of key-clicks. For instance, with the list of questions within the FAQ page, the user can navigate the said list of questions with the arrow keys (via keystrokes) within the keyboard.

Overall, the most used input for the application will be mouse clicks due to the application relying on the use of buttons. However, there are other forms of inputs which are used to a much smaller degree. Such as keystroke inputs from the user’s keyboard.

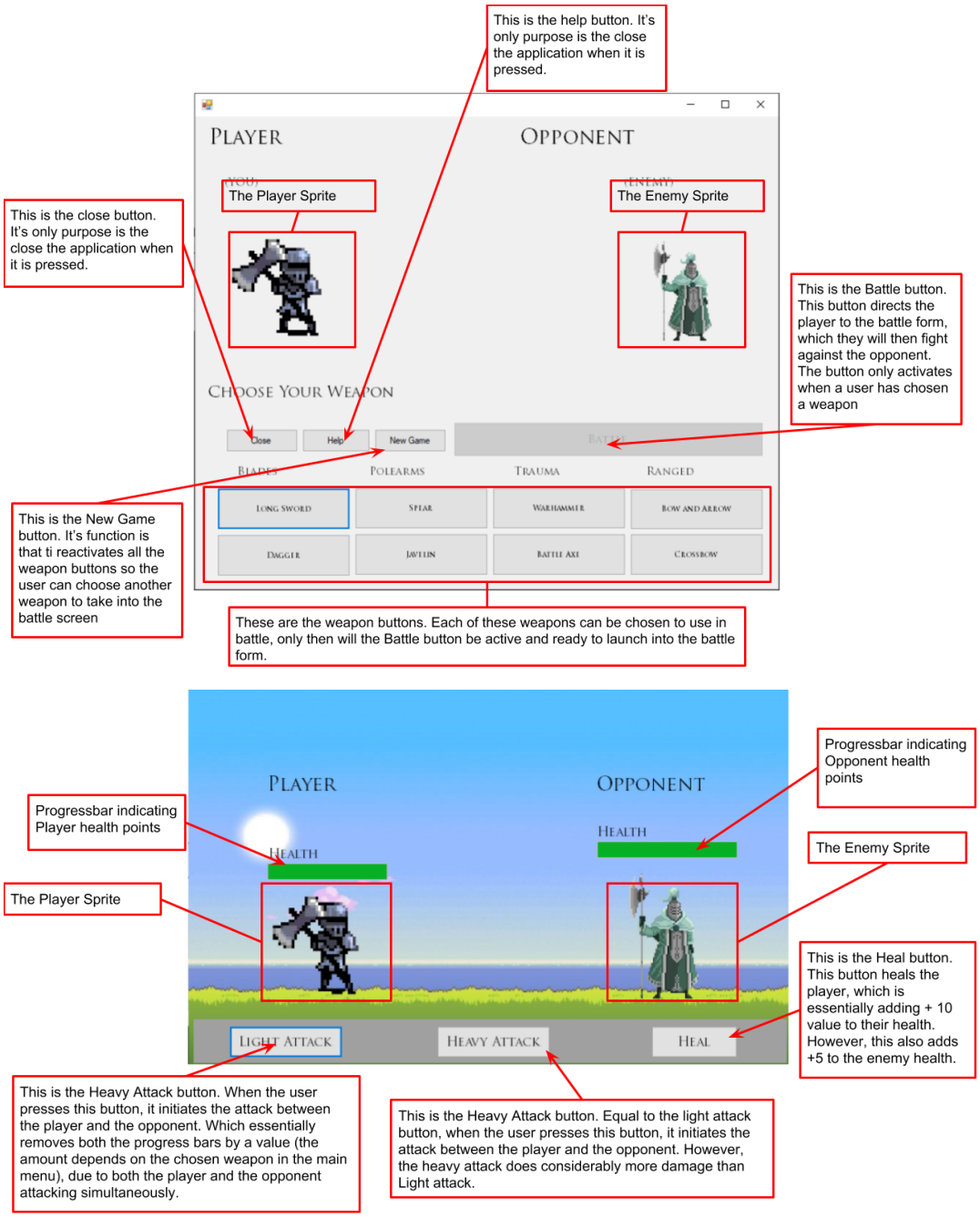
Provide a storyboard showing the main processes/actions of the game

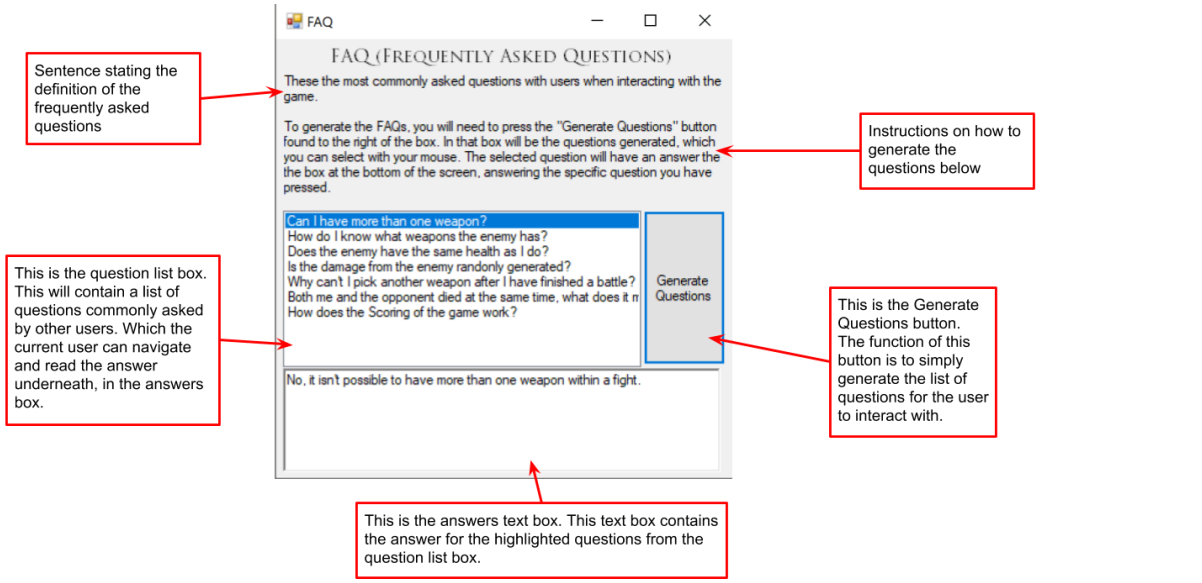


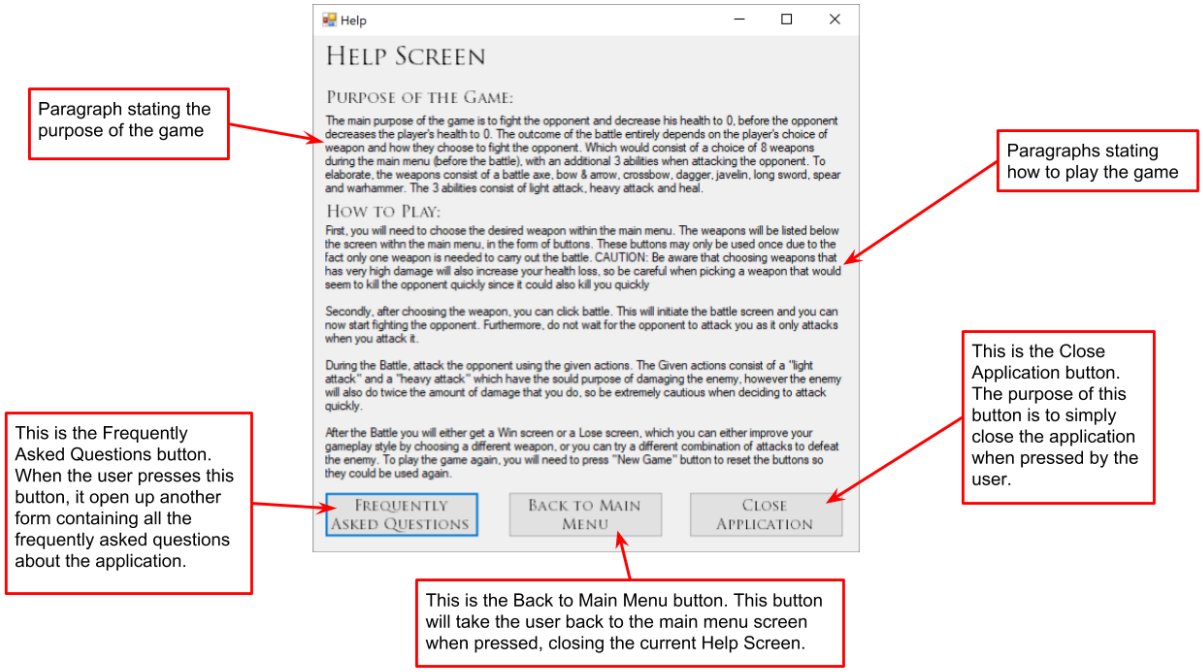
In the storyboard above, it shows the process that a user will have during the video game created. To elaborate, the illustration above shows the procedure the user will take when launching the application, navigating through the different forms found within the video game.

Essentially, it's a procedural flow through the planned application that the average user would be experiencing through. Accessing all the available forms, finding out key information through the informative screen and then carrying out the purpose of the video game, which is to battle against an opponent and (hopefully) win.

Provide annotated screenshots showing the main graphical components in the game




FAQ
FAQ (FREQUENTLY ASKED QUESTIONS)
These the most commonly asked questions with users when interacting with the game.
To generate the FAQs, you will need to press the "Generate Questions" button found to the right of the box. In that box will be the questions generated, which you can select with your mouse. The selected question will have an answer the the box at the bottom of the screen, answering the specific question you have pressed.
Can I have more than one weapon?
How do I know what weapons the enemy has?
Does the enemy have the same health as I do?
Is the damage from the enemy randonly generated?
Why can't I pick another weapon after I have finished a battle?
Both me and the opponent died at the same time, what does it m
How does the Scoring of the game work?
Generate Questions
No, it isn't possible to have more than one weapon within a fight.
Sentence stating the definition of the frequently asked questions
Instructions on how to generate the questions below
This is the question list box. This will contain a list of questions commonly asked by other users. Which the current user can navigate and read the answer underneath, in the answers box.
This is the Generate Questions button. The function of this button is to simply generate the list of questions for the user to interact with.
This is the answers text box. This text box contains the answer for the highlighted questions from the question list box.


Help
HELP SCREEN
PURPOSE OF THE GAME:
The main purpose of the game is to fight the opponent and decrease his health to 0, before the opponent decreases the player's health to 0. The outcome of the battle entirely depends on the player's choice of weapon and how they choose to fight the opponent. Which would consist of a choice of 8 weapons during the main menu (before the battle), with an additional 3 abilities when attacking the opponent. To elaborate, the weapons consist of a battle axe, bow & arrow, crossbow, dagger, javelin, long sword, spear and warhammer. The 3 abilities consist of light attack, heavy attack and heal.
HOW TO PLAY:
First, you will need to choose the desired weapon within the main menu. The weapons will be listed below the screen withn the main menu, in the form of buttons. These buttons may only be used once due to the fact only one weapon is needed to carry out the battle. CAUTION: Be aware that choosing weapons that has very high damage will also increase your health loss, so be careful when picking a weapon that would seem to kill the opponent quickly since it could also kill you quickly
Secondly, after choosing the weapon, you can click battle. This will initiate the battle screen and you can now start fighting the opponent. Furthermore, do not wait for the opponent to attack you as it only attacks when you attack it.
During the Battle, attack the opponent using the given actions. The Given actions consist of a "light attack" and a "heavy attack" which have the sould purpose of damaging the enemy, however the enemy will also do twice the amount of damage that you do, so be extremely cautious when deciding to attack quickly.
After the Battle you will either get a Win screen or a Lose screen, which you can either improve your gameplay style by choosing a different weapon, or you can try a different combination of attacks to defeat the enemy. To play the game again, you will need to press "New Game" button to reset the buttons so they could be used again.
FREQUENTLY ASKED QUESTIONS
BACK TO MAIN MENU
CLOSE APPLICATION
Paragraph stating the purpose of the game
Paragraphs stating how to play the game
This is the Close Application button. The purpose of this button is to simply close the application when pressed by the user.
This is the Frequently Asked Questions button. When the user presses this button, it open up another form containing all the frequently asked questions about the application.
This is the Back to Main Menu button. This button will take the user back to the main menu screen when pressed, closing the current Help Screen.

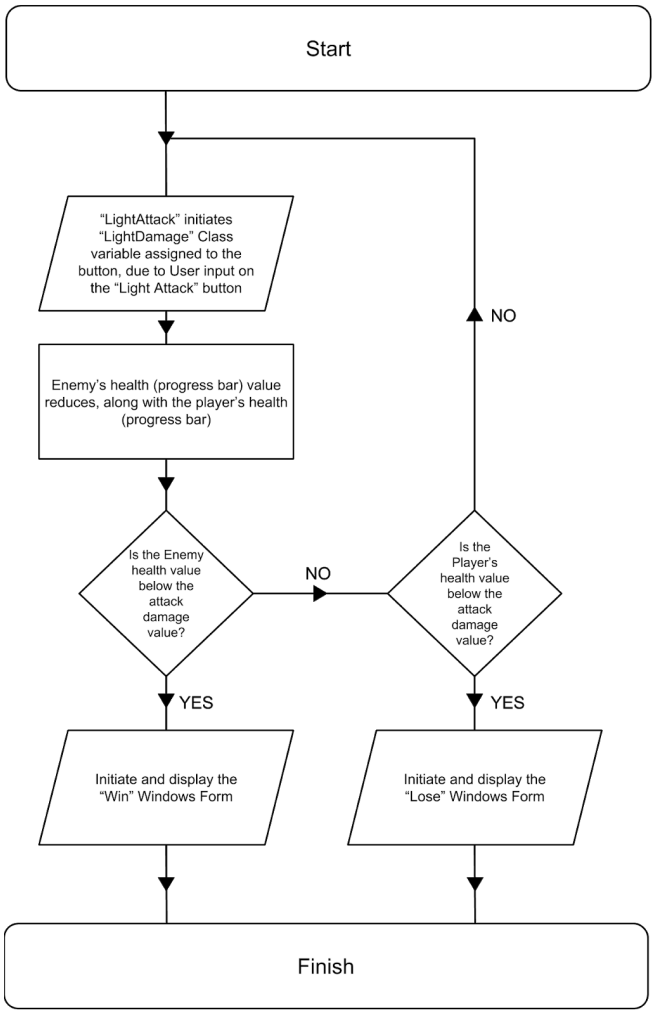




Describe one major event handler. Also provide a flowchart of this event handler.

Within my program, there will be many event handlers which will have the purpose of initiation specific parts of the source code, which will be used to carry out a specific function or purpose. Usually, the event handlers will have very simple source codes, which carries out a simple procedure.

However, there are some event handlers which contain more complicated code and is much more important to the core gameplay. For instance, one a major event handler which will be the core to the gameplay as a whole, is the “Attack” button, specifically the “Light Attack” button (although the Heavy Attack button has the same functions, therefore making it another major event handler.)



The major event handler shown above is the “Light Attack” button found within the Battle windows form, when the player is playing against the AI opponent. In the illustration above, it shows the process of the event handler when it is activated through user input.

Firstly, the event handler is initiated through the click of “btnLightAttack”. Which will then initiate the methods of the class and its properties. Which is referenced as “LightDamage” within the “AttackingWeapon” abstract class, then the value (i.e. “20”) will be used to remove the enemy’s progress bar (indicating health). With the player’s progress bar decreasing by twice the amount, implying to the user that the enemy deals twice the damage as the player.

Continuously, the programme will carry out a decision which will be based off whether the enemy's health value (progressbar, which ranges from 0-100) is below the value of damage (i.e. 20) that the player currently has active.

If the value of the enemy's progress bar is below the value of the "weapon damage" that the player currently has active, the programme will then open the "Win" windows form. Indicating that the player has won the battle against the opponent, giving them the score of victory.

However, If the value of the enemy's progress bar isn't below the "weapon damage" value which the player has active, the programme will then switch to another decision to carry out.

With the new decision, the programme will then check if the player's progress bar value (indicated as health) is below the damage value of the enemy's weapon. If the player's progress bar value isn't below the damage value of the enemy's weapon, then it would loop back to the start of the initiation.

However, if the player's progress bar is below the damage value of the enemy's weapon, then the programme will open up the "Lose" windows form. Indicating to the player that they have lost the battle, losing out on the score of that battle.

Describe how you will use a List, or similar collection, of objects.

The way in which a list will be used within the program is through the use of FAQs (Frequently Asked Questions). Which I simply used to store text in the form of questions, which will be answered by user interaction on the list, triggering if statements which contains text in the form of answers. Essentially, this list will only consist of text and can old be interacted in the form of a "SelectedIndex".

For instance, there will be questions which will be listed within a listbox, which the user will see within the FAQ screen. These questions are simple questions which are commonly asked by users when interacting with the program. With the use of "SelectedIndex", if statements will be created with each "SelectedIndex".

With the collection of objects, I will have a classification of 8 weapons, which will all come under the "AttackingWeapon" abstract class within the class diagram. These 8 weapons will inherit and polymorph two different types of fields: "LightDamage" and "HeavyDamage". Which will have their own respectable values, while still inheriting the same fields from the same abstract class. Hence the polymorphism aspect of the code, more specifically the overriding aspect of polymorphism.

Overall, the list of objects will essentially be text, which will be used as questions within the programme. That will be answered through the means of if statements and user interaction with each individual question.

The collection of objects will be the vital part of the application, since it is what will make the gameplay possible. The reason being that the weapons are the key part of the video game, since it is focused around the selection of weapons and the use of them to defeat the opponent within the battle.

Task 2 - **IMPLEMENTATION** *(Unit 14 P2, P4; Unit 15 P2, P4; Unit 22 P4)*

Task description - *Please implement your game, using the following criteria as a guide.*

- *Implement your game as a .NET C# Windows Forms application, using good variable naming, good indentation of code, and good use of data structures such as Lists*

- *Include source code comments that explain the following:*
- *Demonstrate good use of IF and looping logic*
- *Demonstrate valid declaration, initialisation and assignment of variables*
- *Demonstrate data/input validation*
- *Demonstrate error handling and reporting (this may be simply messages to the user)*
- *Show how you have found and fixed bugs*
- *Show that your form(s) make good use of Windows Forms Controls, and properties*
- *Show the Class Diagram for your application*

And thereby demonstrating the following:

- *Predefined objects and functions, screen templates*
- *Using the MS Visual Studio IDE, and Class Diagram*
- *Using variables: global, local, static, class, object instance*
- *Also by adding classes, properties, inheritance, polymorphism, abstraction.*

Implementation

The evidence for this task will be the creating and implementation of my video game. The evidence of each criteria will be visible within the source code comments which I have created, which was used to both ease development and give enough evidence for the guide above. Basically, the above criteria will be evident within the source code comments.

Task 3- ON-SCREEN HELP *(Unit 14 P6; Unit 15 P6; Unit 22 P6)*

Task description - *Create on screen help. This help should include:*

1. *The purpose of game,*
2. *The operation of game, ie how to play, how to win, how the scoring works etc*
3. *FAQs (Frequently Asked Questions)*

Your help feature should be implemented on one or more separate forms and use appropriate controls (eg WebBrowser displaying PDF file).

Task 4 - TECHNIQUES USED *(Unit 14 M2; Unit 15 M2)*

Task description - *Please provide the following:*

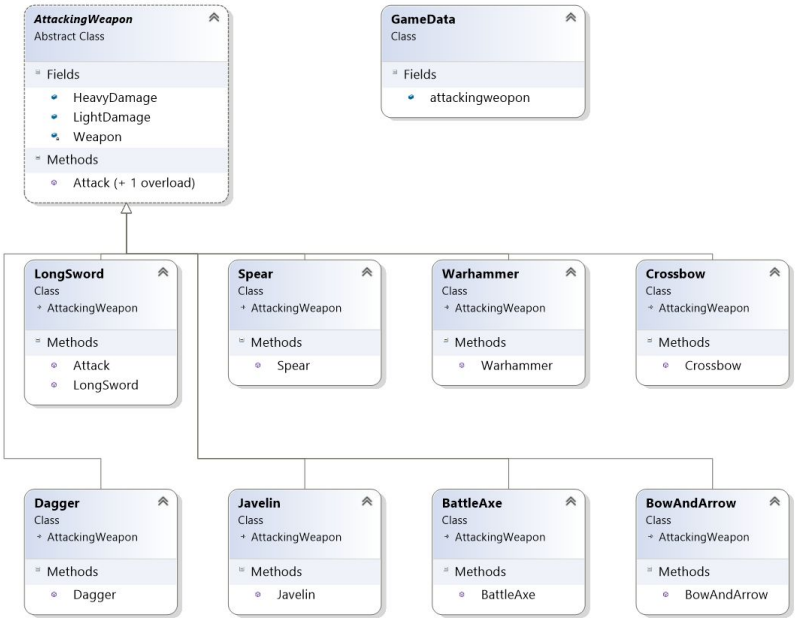
1. *A discussion, with screenshot(s), of how the MS Visual Studio Class Diagram tool has helped you in your development efforts*
2. *A discussion, with screenshot(s) of how you have found and fixed a specific bug using the MS Visual Studio debugger*
3. *A discussion of your reasons why you have used built-in .NET classes and/or methods (eg the Random class)*

4. *A discussion of some forms controls that you found especially useful, including your reasons.*

A discussion, with screenshots, of how the MS Visual Studio Class Diagram tool has helped you in your development efforts

Within my program, the use of the MS Visual Studio Class Diagram tool has aided me in the development and creation of the entire application, due to the fact it was such an integral part to storing different weapon classes. Essentially, the class diagram helped me in the creation, distribution and use of different types of classes within the game.

Additionally, the class diagram also helped me with the inheritance and polymorphism for the classes, not only with encapsulation. For instance, the main abstract class what has two key fields inherited and polymorphisms, is the “AttackingWeapon” abstract class.



In the screenshot above, the main abstract class shown above is the “AttackingWeapon” class found within the class diagram. In the illustration above, it shows the different fields and methods which it contains. Which is inherited to the other weapon classes classes, indicated with the inheritance line.

The first reason why it helped me with the development of my programme, is mostly due to the GameData class. Which was created and used to store and transfer data over different forms within the application, instead of having the data only in one form. The field which carries this out is referenced as “attackingweopon”.

Essentially, the attackingweopon field will store and transfer data which will be based off which weapon the user has selected on the main menu screen. Transferring the said values and other data from any of the classes under “AttackingWeapon” to the battle screen, to be used as the active weapon when fighting against the opponent.

Another reason is the class diagram helped me create a collection of weapons, which inherits the fields from the AttackingWeapon abstract class. Which I would then polymorphisise the fields within each class, giving them a unique “LightDamage” and “HeavyDamage” value.

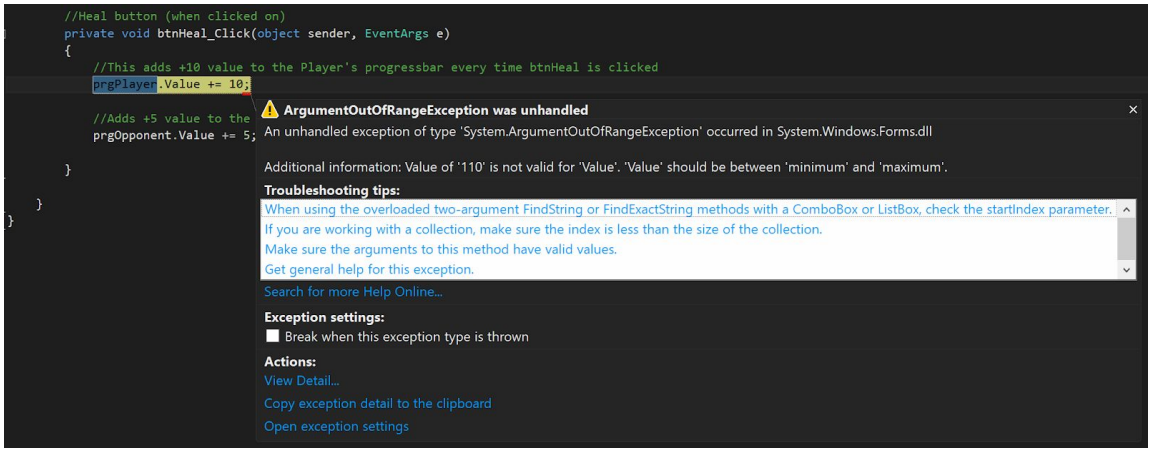
Which I have stored within those individual classes as methods, due to the fact it is going to be referenced and used within the source code within the “FightingApp” and “Battle” windows forms.

A discussion, with screenshots of how you have found and fixed a specific bug using the MS Visual Studio debugger

While testing out my application, I have come across a bug which was caused by the “heal” button. Essentially, the error occurred whenever I pressed the heal button while the player’s progressbar is full or almost full.



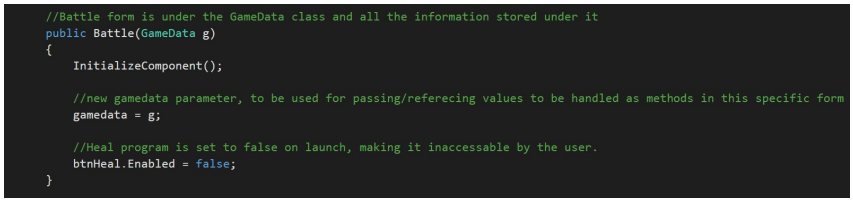
When I clicked the button shown above, the programme stopped responding and then crashed gave me an error message. The error message is shown below, with a breakpoint preset to indicate where the error occured.



After reading the error message and finding out why the error has occured, I opened the debugger to inspect the code close. With the debugger, I set a breakpoint where the error occured. With this information I could see where the data is being transferred to, and the value of the said data.



Therefore, with that knowledge I decided to disable the heal button when the programme is launched, due to the fact the player will start with full health initially. The way I have done this is created a “btnHeal.Enabled = false;” declaration under the form’s field. As shown in the screenshot below.



Then, I created a declaration so that the button will be enabled whenever the player's progress bar has decreased. The way I done this is shown in the screenshot below, which I have added "btnHeal.Enabled = true;" in the else statement for the light damage and heavy damage button.

```
//Enable the heal button
//Reduce player health with predefined weapon damage value
//The damage reduces is the same field to what the player chose for "attackingweopon", but with extra damage
else
{
    btnHeal.Enabled = true;
    prgPlayer.Value -= gamedata.attackingweopon.LightDamage * 2;
}
```

```
//Enable the heal button
//Reduce player health with predefined weapon damage value
//The damage reduces is the same field to what the player chose for "attackingweopon", but with extra damage
else
{
    btnHeal.Enabled = true;
    prgPlayer.Value -= gamedata.attackingweopon.HeavyDamage * 2;
}
```

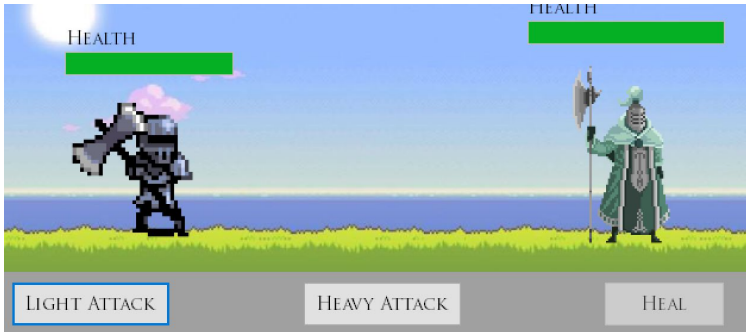
Lastly, I added a final declaration under the "btnHeal_Click" void, where it would essentially disable the button when the player's health (progress bar) is above the value of 95. The reason for this number is due to the fact that weapon damage is scaled in multiple of 5s. Also, I have added a message box which will warn the user that the healing ability has reached its limit.

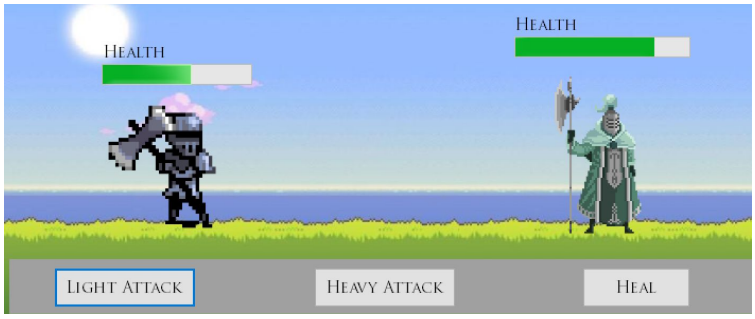
```
//Heal button (when clicked on)
private void btnHeal_Click(object sender, EventArgs e)
{
    //This adds +10 value to the Player's progressbar every time btnHeal is clicked
    prgPlayer.Value += 10;

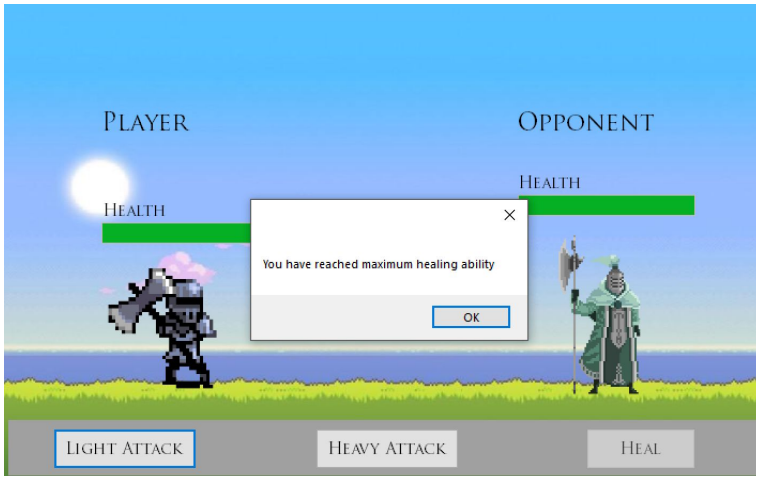
    //Adds +5 value to the opponent's progressbar every time btnHeal is clicked
    prgOpponent.Value += 5;


    //If the progress bar value is above 95, disable btnHeal to prevent the application from crashing
    //Indicate user with a messagebox with a warning stating why btnHeal is disabled
    if (prgPlayer.Value >= 95)
    {
        btnHeal.Enabled = false;
        MessageBox.Show("You have reached maximum healing ability");
    }
}
```

After implementing the code, I tested the application to see if the bug was resolved and that the application had an element of error handling and reporting. As shown below, is the progress of testing the application if the declarations work, and if it prevented the game from crashing.







After testing the program, I came to the conclusion that the added declarations did indeed function as intended, and that the program can no longer crash from the user adding too much to the progress bar value.

Overall, as illustrated on the screenshots shown above, I have found a bug within the application which, when the heal button is pressed, the progress bar would be given more value than is possible to contain. Causing the application to crash.

The way in which I have fixed it is by only enabling the button when the player's progress bar loses in value, disabling the button when the value is full and warning the user that the value has reached its limit and can no longer be added.

A discussion of your reasons why you have used built-in .NET classes and/or methods (eg the Random class)

The built-in .NET classes using within my program consist of "Thread", "List", "Application" and "MessageBox". The reason I have used the predefined classes within the program is due to their ability of manipulating and referencing the system data, which I can use to my advantage when creating specific sequences of programme code.

To elaborate, the "Thread" class is essentially a class which specifies a delegate which allows an object to be passed to the thread, only when the thread is started however. The way which I have used this class is through the use of a while loop, which decreases the opacity of the program and then increases it after a 20/40 millisecond gap. Essentially, I used threat in order to have a millisecond gap between the loop, so it gives the application a visual indication of a hit.

The "list" class is a class which essentially contains a list of objects, which can be accessed via an index. To elaborate, the list class provides multiple methods to search, sort and even manipulate lists. The way I have used this class is to create a new string list of questions, which are essentially text. For example, in the source code I created a variable for the questions using the list class, which I have written as "List<string> questions = new List<string>();".

The "application" class simply provides methods (static) and other properties in order to manage the application. For instance, methods which starts or stops and application, processes windows messages and even properties to get information about the said application. I have used this class to only close the application when a specific button is pressed, for example the "Close" button found within the "From1" form.

Lastly, the “MessageBox” class is essentially a class which displays a message window, otherwise known as a dialog box, which shows a message to the user. Typically, a messagebox can contain text, button and even symbols which can be used to either inform or instruct the user. The way I have used the message box within the program, was to inform the user that the “healing” button is no longer usable.

Additionally, as mentioned previously, I have created a “gamedata” class in order to store all the properties under the “AttackingWeapon” abstract class and all its inheriting classes. I have used this class to essnetially transfer the data across different forms, specifically the “FightingApp” and “Battle” form. Carrying the “Heavy Damage” and “Light Damage” fields and their properties.

Essentially, the use of pre-defined .NET classes helped with the use and declaration of certain methods and other properties, in order to create unique and other specific sequences within the programme.

A discussion of some forms controls that you found especially useful, including your reasons.

With the development phase of the application, there were a couple of form controls which I found particularly useful due to the ease of development that they bring.

Firstly, and most importantly, was the use of buttons. This form control has helped me due to the fact I could use the buttons to access different sequences of programme code, use buttons to attack the enemy and even use them to access different forms. Essentially, the buttons within the form controls has aided me to create the foundation for the application. Meaning that, without the use of buttons, the application would of been impossible to create without it being a console application.

Secondly, is the use of progress bars. The way I have used this is really simple, and it is used as a health indication for the player. I found this useful since it is a much more visually pleasing method of indicating health, rather than a simple label. Essentially, progress bars gave me a graphical method of indicating health to the player, rather than using only text.

However, labels, textboxes and listboxes also proved to be useful due to the fact they give the user the informative feedback required to access the specific parts of the program, which they would otherwise wouldn’t know without assistance. Essentially, labels, textboxes and listboxes aided me with the informative aspect of the application, giving the user information on how to use and navigate through the programme.

Additionally, panels was useful due to the fact it made a much more arranged application. Essentially, panels only benefited the aesthetics of the programme, helping with the arrangement of buttons and other form controls. However, panels provided no other benefits apart from the visuals of the programme.

Lastly, picture boxes also aided with the visual aspect of the programme. They helped with the implementation of sprites within the programme. Which was used to differentiate the player from the opponent, while also having a visual of the player and opponent’s character.

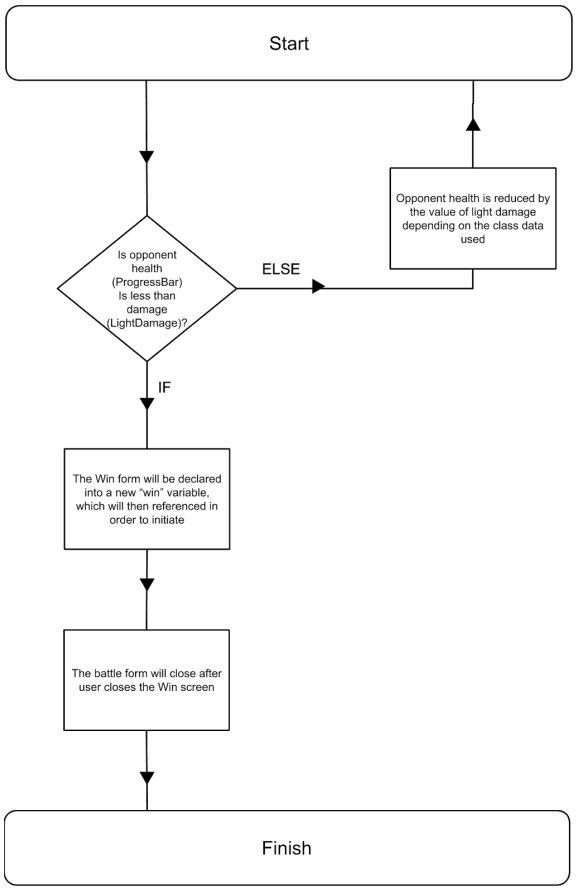
Overall, the form controls mostly aided with the visuals of the application. However, in the case of buttons and text based controls (labels, textboxes and list boxes), helped create and link the rest of the programme. Giving the user the ability and the knowledge of navigation throughout the programme.

Task 5 - TECHNICAL DOCUMENTATION *(Unit 22 P7; Unit 14 M4; Unit 15 M4)*

Task description - *Please provide the following technical documentation:*

- *Two syntactically correct flow charts showing if and looping logic for the algorithms that you have designed.*
- *Submit a data dictionary showing clearly all variables, data types, and usage*
- *Submit your class diagram (from MS Visual Studio)*

<u>Two syntactically correct flow charts showing if and looping logic for the algorithms that you have designed.</u>



This if statement is used within the Battle forum, used to indicate whether the player/opponent has a progress bar (health) value below the damage value of the weapon, indicating whether the user has won, lost or the battle is still occuring.

Firstly, the if statement is initiated through the click of either “btnLightAttack” or “btnHeavyAttack”. Which, as mentioned previously, will then initiate the methods of the class and its properties.

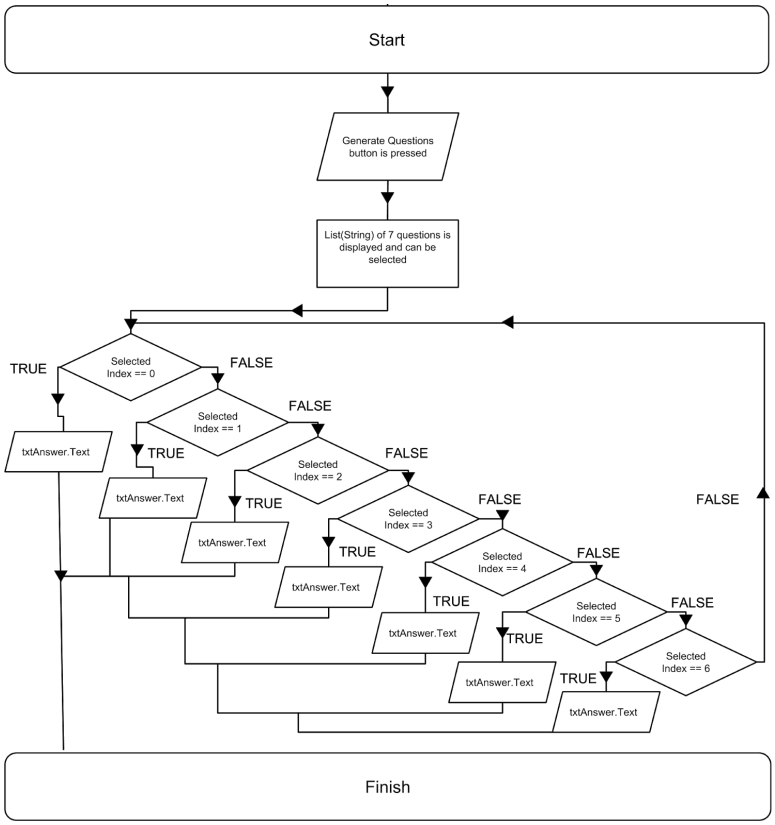
Referenced as “LightDamage”/”HeavyDamage” within the “AttackingWeapon” abstract class, then the value (i.e. “20”) will be used to remove the enemy’s progress bar (indicating health). With the player’s progress bar decreasing by twice the amount, implying to the user that the enemy deals twice the damage as the player.

If the value of the enemy’s progress bar isn’t below the “weapon damage” value which the player has active, the programme will then carry out the action of reducing the opponent’s health (progress bar

value) by the value of the damage (LightDamage/HeavyDamage). Then it will go back to the start, which requires the user to press the button again in order to initiate the if statement.

With the new decision, the programme will then check if the player's progress bar value (indicated as health) is below the damage value of the enemy's weapon. If the player's progress bar value is below the damage value of the enemy's weapon, then it would open a new Win form, which would be initiated through the "win" variable.

If the value of the enemy's progress bar is below the value of the "weapon damage" that the player currently has active, the programme will then open the "Win" windows form. Indicating that the player has won the battle against the opponent.



The flowchart shown above shows the looping logic that I have used within the program, however "If statements" were used in order to create this loop. In the flowchart above, it shows the if statement loop that I have implemented within the program. This if statement is used within the FAQ forum, used to answer the questions which the user has selected from the list of questions.

Essentially, the if statement shown above is the Answer text box found within the FAQ windows form, when the player is navigating through help to find out about the game. In the illustration above, it shows the process of the if statements when it is activated through user interaction.

Firstly, it is initiated through the click of "Generate Questions". Which will then initiate the text, in the form of questions. Which is referenced as "Questions" within the "btnFAQ" button, then the questions will be displayed, with the listbox displaying all the questions listed. Continuously, the programme will carry out a decision which will be based off whether the question currently selected (SelectedIndex) is true or false.

If the question that is listed is currently selected, through the use of a mouse click, the programme will then initiate the pre-written text in the answer textbox. Displaying the answer on the assigned text box, giving the user feedback on the desired question.

However, if the current question isn't selected, the programme will then transfer to the next decision, which would then be "SelectedIndex ==2". With the new decision, the programme will carry out the same operation as the previous decision. Which will be identical for the rest of the decisions within this if statement.

However, the final decision is an exception, due to the fact the "false" value would direct the programme to the beginning of the sequence, essentially similar to a loop. Which the programme will start from "SelectedIndex == 1". Which would go through the same sequence repeatedly, until the user closes the form which contains the list.

Submit a data dictionary showing clearly all variables, data types, and usage

Field/Variable Name	**Data Type**	**Description of use**
attackingweopon	Object	This variable contains the data to be transferred from one form to another. For instance, containing the value "Heavy Damage", which is 35. From the "LongSword" class, which inherits the "HeavyDamage" field from the AttackDamage abstract class. Which is what the gamedata is referenced to.
gamedata	Object	This variable is used to store the data from the "attackingweopon" object within the form, and to reference the said data within the said form.
HeavyDamage	int	Stores the maximum damage value that the weapon currently has stored within the class. This field inherited and polymorphed by the classes which has an inheritance connection to the "AttackingWeapon" abstract class.
LightDamage	int	Stores the minimum damage value that the weapon currently has stored within the class. This field is inherited and polymorphed by the classes which has an inheritance connection to the "AttackingWeapon" abstract class.
Opacity	Double	Used to change the opacity (transparency) of the form. For example, this variable is referencedwhen the player attacks the enemy, causing the enemy to hit back.
questions	Local variable	Used as the variable to the list which contains the "questions" within the FAQ form.

Weapon	int	This inherits and creates (polymorphs) the fields from the "AttackingWeapon", creating a new value to the pre-existing field. An example of a weapon field is the "LongSword" integer found in the "LongSword" Class.
frm	Local variable	Used for the declaration of a form, either to reference the form to close or open when declared. An example of a frm variable being used is the "faqfrm", which is used to reference the form to open when the button is clicked.
prg*Name*.Value	int	The value of the progress bar, which ranged from 0-100. This is used within the battle form to indicate damage, which the value of the "HeavyDamage" or "LightDamage" integers are used to subtract the progress bar value. For example, the "prgPlayer.Value" is a field within the game, which is used to indicate the player's health. Which is reduces when the opponent attacks the player and increased when the player heals.
btn*Name*.Enabled	bool	Set to "true" when available to be clicked by the user, set to "false" to prevent the user from accessing the button and the properties within it. For instance, one example of this field within the program would be "btnSword.Enabled". The function of this variable is to essentially determine whether a button is active or not, depending on what the user has done in order to activate the declaration.

Submit your class diagram (from MS Visual Studio)

The class diagram within the MicroSoft Visual Studio programme is going to be evident with the submission of the application, rather than the documentation. Although, there is screenshot evidence of the class diagram above within the documentation.

Task 6 - OBJECT ORIENTED DESIGN *(Unit 15 P3)*

Task description - *Please produce the following evidence.*

- *Your Class Diagram MS Visual Studio must show the following, clearly:*
  - *Classes and their dependencies and inheritances*
  - *Identification attributes (properties/fields) and their scope (eg: private, public)*
  - *Methods and their scope*
  - *Overloading and overriding of methods inheritance; aggregation; association*
- *Explain your use of any pre-defined classes*
- *Show clearly your class constructors*

Explain your use of any predefined classes

As mentioned before, the built-in .NET classes using within my program consist of “Thread”, “List”, “Application” and “MessageBox”. The reason I have used the predefined classes within the program is due to their ability of manipulating and referencing the system data, which I can use to my advantage when creating specific sequences of programme code.

Task 7 - DETERMINE DATA TYPES *(Unit 22 M1)*

Task description - *Your program code will contain variable declarations that also include the data type of the variable in question.*

For each variable declaration provide a comment above explaining clearly what the variable is to be used for, and justifying the data type used.

Example:

```
// Create a variable to hold the current score of our main warrior.
// It is declared as integer because only whole numbers are valid for score.
int warriorscore = 0;
```

Determine data types

As mentioned previously, the evidence for such criteria above will be commented within the source code. Indicating the sufficient evidence required for the listed criteria above.

Task 8 - SELECTION & ITERATION METHODS *(Unit 22 M2)*

Task description - *Your program code will contain a number of if (decision) blocks, and a number of loops. For each instance of a decision (if), and a loop, please provide comprehensive in-code comments that explain the purpose.*

Example (if)

```
// If there are no questions available in list
//  of questions, warn the user.
if (questions.Count == 0)
{
    sbStatus.Text = "There are no questions in the database";
}
```

Example (loop)

```
// Loop (iterate) through all questions in the
//  list of questions. Return from this method
//  if text of new question matches a current
//  question in list.
// Is question already in questions list?
foreach (Question que in questions)
{
   if (txtModelQuestion.Text.Trim() == que.question)
   {
      sbStatus.Text = "That question is already in the database!";
      return;
   }
}
```

Selection & Iteration models

Yet again, as mentioned previously, the evidence for such criteria above will be commented within the source code. Indicating the sufficient evidence required for the listed criteria above.

Conclusion

In conclusion, the game will be a dueling game centered around ancient medieval weaponry, which would have the player focus on the strategic use of the types of damage and when to heal in order to defeat the opponent. Overall, the game that I have designed is relatively simple in code. However, contrary to sophisticated codes, the code carries out its intended purpose well and has a good degree of user friendliness and error handling/reporting.